

CafeOS Technical Specification: A Comprehensive Product Architecture for a Next-Generation Cafe Mobile Application

1. Executive Summary and Architectural Vision

This document articulates the complete technical and product specification for **CafeOS**, a mobile application designed to serve as the digital operating system for a premium cafe network. Unlike generic white-label solutions or feature-bloated super-apps, CafeOS is architected with a philosophy of "utilitarian elegance." The primary objective is to reduce the friction between desire (caffeine/sustenance) and fulfillment, while layering in a nuanced, privacy-conscious social texture and a transparent loyalty mechanism.

The overarching design ethos leverages the "Editorial" trend in mobile UI, prioritizing typography, whitespace, and calm interaction models over high-octane gamification.¹ This approach aligns with a user base that values clarity and efficiency. Technically, the system is designed as an iOS-first (but platform-agnostic logic) application, relying on a robust RESTful backend, strict transactional integrity for financial operations, and an optimistic UI pattern on the client side to ensure perceived latency remains near zero.³

The scope of this specification encompasses four distinct but interconnected domains: a multi-modal **Ordering Engine** (Dine-in, Take-away, Delivery), a transparent **Loyalty Protocol** based on RFM (Recency, Frequency, Monetary) analysis, a **Store Intelligence** module handling geolocation and operational status, and a minimal **Social Connection** layer designed to facilitate real-world meetups rather than digital retention.

This is a build-ready document. It moves beyond conceptual broad strokes to define data schemas, strict user flows, edge-case handling, and the specific design tokens required for implementation by a dedicated engineering team.

2. Core Product Mechanics and Functional Pillars

The application's functionality is strictly bounded to prevent scope creep, ensuring that every feature serves a tangible utility. The four core mechanics are defined below with their specific technical constraints.

2.1 The Ordering Engine

The ordering mechanism is the central nervous system of the application. It is not merely a

form for data entry but a state machine that manages inventory reservation, price validation, and fulfillment logistics. The engine supports three distinct fulfillment modes, each triggering unique validation logic at the API level:

- **Dine-in:** This mode creates a digital-to-physical bridge. It necessitates the input of a TableNumber (scanned via QR code or manually entered). Crucially, this mode bypasses packaging fees and delivery surcharges. The inventory check is strictly scoped to the local store's real-time stock, preventing orders for items that physically exist in the catalog but are momentarily depleted at the specific location.
- **Take-away:** This mode introduces temporal logic. It requires a PickupTime slot selection. To prevent kitchen overload, the backend implements "Throttle Balancing," limiting the number of orders acceptable per 15-minute window based on the store's throughput capacity.⁵ This prevents the common failure state where a digital order queue overwhelms the physical barista team.
- **Delivery:** This mode extends the domain beyond the store's walls. It requires DeliveryAddress validation against a geo-fenced polygon defining the store's delivery radius. It also triggers the calculation of dynamic fees and an estimated time of arrival (ETA) that synthesizes driver availability (via third-party logistics APIs) and kitchen prep time.⁶

2.2 The Transparent Loyalty System

The loyalty mechanics reject the opaque "black box" algorithms often found in modern apps. Instead, CafeOS implements a transparent "Earn & Burn" model. This builds trust by making the value exchange explicit.

- **Accrual Logic:** Users earn a fixed rate (e.g., 10 Points per currency unit). Points are accrued only upon the COMPLETED state of an order, preventing fraud from cancelled or refunded transactions.
- **Redemption Mechanics:** The system treats Rewards as a distinct inventory class. A user exchanges points for a RewardObject (e.g., "Free Pastry"), which acts as a 100% discount voucher applied to a specific SKU.
- **Voucher Architecture:** The system supports a flexible voucher engine capable of handling percentage-based discounts, fixed-value deductions, and conditional logic (e.g., "Buy X, Get Y"). These are validated server-side to prevent stacking exploits.

2.3 Store Intelligence and Location Services

This module handles the geospatial data and the operational status of the physical network.

- **The Store Registry:** A master database of all locations, containing immutable metadata (Latitude/Longitude, Address) and mutable status flags.
- **Dynamic Status Logic:** The application must handle complex status states beyond simple "Open/Closed." It includes "Busy" flags calculated via order throughput and "Delivery Unavailable" flags if the 3PL (Third Party Logistics) network is saturated.⁷
- **Geospatial Sorting:** The client utilizes the device's location services to sort stores not just by linear distance, but by "functional distance"—prioritizing open stores and those

within delivery range over closer but closed locations.

2.4 Light Social Connection

The social feature is designed as a utility for coordination, not content consumption. It avoids feeds, likes, or algorithmic engagement loops.⁸

- **Privacy-First Presence:** The "Connect" feature allows users to broadcast their presence at a specific store ("I'm at Downtown Cafe") only to a mutually accepted "Inner Circle" of friends. This state has a strict Time-To-Live (TTL), automatically expiring after a set duration or when the user leaves the geofence, ensuring safety and preventing stalking risks.⁹
- **Peer-to-Peer Gifting:** The primary interaction is transactional benevolence—buying a product for a friend. This creates a "Gift Object" in the recipient's wallet, redeemable at the specific location.

3. Data and Domain Model

The integrity of the application relies on a robust data schema. The following models define the entities, their relationships, and the ownership of data between client and server.

3.1 User Entity

The User entity is the root of the identity graph. It is server-owned but partially cached on the client for performance.

Field	Data Type	Required	Description
id	UUID	Yes	Unique immutable identifier.
auth_id	String	Yes	Foreign key to Identity Provider (Auth0/Firebase).
phone_number	String	Yes	Primary key for user uniqueness and delivery contact.
email	String	Yes	Used for receipts and marketing communication.
display_name	String	Yes	User's preferred name for greetings/social.
loyalty_tier	Enum	Yes	BRONZE, SILVER, GOLD. Derived from lifetime_points.
current_points	Integer	Yes	Current spendable balance.

created_at	Timestamp	Yes	Account creation date.
------------	-----------	-----	------------------------

Relationships:

- One-to-Many with Order.
- One-to-Many with SavedAddress.
- Many-to-Many with User (via Connection table).

3.2 Store Entity

The Store entity represents the physical node in the network.

Field	Data Type	Required	Description
id	UUID	Yes	Unique store identifier.
name	String	Yes	Display name (e.g., "CafeOS Downtown").
geo_location	Point	Yes	(Lat, Long) for distance calculations.
timezone	String	Yes	IANA timezone ID (e.g., "America/New_York").
is_active	Boolean	Yes	Global switch to hide/show store.
status	Enum	Yes	OPEN, CLOSED, BUSY, PAUSED.
tax_rate	Decimal	Yes	Local tax rate for calculations.

Relationships:

- One-to-Many with Order.
- One-to-One with Inventory (a dynamic table mapping SKU IDs to counts).
- One-to-Many with StoreHours.

3.3 Menu and Product Architecture

To support the complexity of cafe customization (e.g., "Latte with Oat Milk, Extra Hot, No Sugar"), the data model uses a nested modifier architecture.¹¹

MenuItem Model:

Field	Data Type	Required	Description
id	UUID	Yes	Product identifier.
name	String	Yes	e.g., "Cappuccino".
base_price	Integer	Yes	Price in cents (to avoid floating point errors).
category_id	UUID	Yes	Link to Category.
modifier_groups	Array	No	List of ModifierGroup

			IDs linked to this item.
--	--	--	--------------------------

ModifierGroup Model:

Field	Data Type	Required	Description
id	UUID	Yes	e.g., "Milk Selection".
min_selection	Integer	Yes	0 = Optional, 1 = Mandatory.
max_selection	Integer	Yes	1 = Radio Button, >1 = Checkbox.
options	Array	Yes	List of ModifierOption objects.

ModifierOption Model:

Field	Data Type	Required	Description
id	UUID	Yes	e.g., "Oat Milk".
name	String	Yes	Display name.
price_delta	Integer	Yes	Additional cost in cents (e.g., +80).
is_available	Boolean	Yes	Controlled by store inventory.

3.4 Cart and Order Models

The Cart is a transient client-side state that synchronizes to the server, while the Order is the immutable record of a transaction.

CartItem Model:

Field	Data Type	Description
id	UUID	Temporary ID for UI tracking.
product_id	UUID	Reference to catalog item.
quantity	Integer	User selection.
modifiers	Array	List of selected ModifierOption IDs.
hash	String	SHA-256 hash of (Product + Modifiers) to merge duplicates.

Order Model:

Field	Data Type	Ownership	Description
id	UUID	Server	Unique Order ID.
user_id	UUID	Server	Customer link.
store_id	UUID	Server	Fulfillment location.

state	Enum	Server	PENDING, CONFIRMED, PREPARING, READY, COMPLETED, CANCELLED.
type	Enum	Client	DINE_IN, TAKEAWAY, DELIVERY.
items	JSON	Server	Snapshot of items at time of purchase (preserves price/names).
total_amount	Integer	Server	Final charge in cents.
payment_id	String	Server	Reference to Stripe/Gateway transaction.

3.5 Connection Model (Social)

This model manages the graph between users.

Field	Data Type	Description
initiator_id	UUID	User who sent request.
recipient_id	UUID	User receiving request.
status	Enum	PENDING, ACCEPTED, BLOCKED.
created_at	Timestamp	Date of connection.

4. User Flows: Detailed Narrative and Logic

The following section outlines the critical paths through the application. Each flow is detailed with entry points, step-by-step actions, and handling of edge cases.

4.1 First App Launch (Cold Start / Guest Mode)

Entry Point: The user taps the app icon for the first time.

Goal: Reduce time-to-value. Avoid "Login Walls." Allow exploration.

1. **Splash & Initialization:** The app displays a minimal logo on a solid background (Espresso color). In the background, the app initializes the API client and checks for a stored session token.
2. **Contextual Permission Request:** Instead of a generic iOS system prompt, the app displays a calm, designed modal: "CafeOS needs your location to show you the nearest coffee." Two buttons: "Allow Location" (Primary) and "Browse Manually" (Secondary).
 - *Decision Point:* If the user taps "Allow," the OS system prompt is triggered. If

"Browse Manually," the user is taken to a city selector list.

3. **The Store Feed (Home):** The user lands on the main "Menu" tab.
 - *State:* The app is in "Guest Mode." The user can browse the full menu, see prices, and view store details.
 - *Edge Case:* If the user taps "Add to Cart" or "Favorites," the "Login/Signup" flow is triggered. This is "Just-in-Time" authentication.

4.2 Login / Signup Flow

Entry Point: Triggered by an action requiring identity (e.g., Checkout, Profile access).

Methodology: Phone Number (OTP) is the primary method. This serves dual purposes: it verifies identity (reducing spam orders) and provides a necessary contact method for delivery drivers.¹

1. **Input Screen:** A clean screen asking, "What's your mobile number?" The input field auto-formats to the local region (e.g., (555) 123-4567).
2. **OTP Verification:** The user receives a 6-digit SMS code. On iOS, the app subscribes to the QuickType bar to suggest the code automatically.
 - *Failure State:* If the code is invalid/expired, shake the input field and show a specific error: "That code expired. We just sent a new one."
3. **Profile Completion (New Users Only):** If the phone number is new, a secondary screen asks for "First Name" and "Email" (for digital receipts).
4. **Restoration:** The user is returned *exactly* to the state they were in before the flow started (e.g., the specific Item Detail screen with their modifiers still selected).

4.3 Browsing the Menu

Design Philosophy: "Editorial." This is a visual experience.

1. **Store Context Header:** The top of the screen clearly states, "Ordering for:". Tapping this opens the Store Selector.
2. **Category Navigation:** A horizontal sticky bar lists categories (Coffee, Tea, Bakery). Tapping one smooth-scrolls the list to that section.
3. **Product List:** Items are presented as rows. Each row contains:
 - Thumbnail image (Left).
 - Title and short description (Center).
 - Base price (Right).
 - *Visual Cue:* If an item is SOLD_OUT in the backend inventory, the row is greyed out and unclickable.

4.4 Placing an Order (The "Order Engine")

This flow varies based on the Fulfillment Type selected.

Scenario A: Take-away (Standard Flow)

1. **Item Configuration:** User selects "Latte." A modal opens.
2. **Modifier Selection:**
 - The user must select a size (Required).

- The user can select a milk (Defaults to "Whole"). Changing to "Oat" updates the displayed price button dynamically (+\$0.80).
- 3. **Cart Addition:** Tapping "Add to Order" validates the selection and closes the modal.
- 4. **Checkout Screen:**
 - The user selects "Pickup" toggle.
 - The user selects "Time": "ASAP (approx 10m)" or a specific future slot.
 - The user confirms payment via Apple Pay or stored card.
 - *Success:* Transition to "Order Tracking" screen.

Scenario B: Delivery (Edge Cases)

1. **Address Input:** At Checkout, selecting "Delivery" prompts for an address.
2. **Geofence Validation:** The app sends the coordinate to the backend.
 - *Edge Case (Out of Range):* The backend returns a 400 error with type DELIVERY_UNAVAILABLE. The UI displays a friendly error: "This store doesn't deliver to that address. Try switching to our location?"
3. **Fee Calculation:** The backend returns an updated OrderSummary object including delivery_fee and service_fee.

Scenario C: Dine-in (QR Flow)

1. **Entry:** User scans a QR code on the table.
2. **Deep Link:** The app opens directly to the Menu for that store, with DINE_IN mode active and table_number pre-filled in the session.
3. **Checkout:** The "Delivery/Pickup" toggle is hidden. The header reads "Dine-in at Table 4." No packaging fees are applied.

4.5 Cart Management

Entry Point: The floating "View Order" button on the Menu screen.

1. **List View:** Items are listed with their full modifier strings (e.g., "Latte / Oat / No Sugar").
2. **Editing:**
 - *Quantity:* Simple Stepper (- 1 +). Reducing to 0 triggers a "Remove Item?" confirmation.
 - *Modifiers:* Tapping the item re-opens the configuration modal.
3. **Conflict Resolution**¹³:
 - *Scenario:* A user adds a muffin. 10 minutes later, before checkout, the store runs out.
 - *Handling:* On entering the Cart, the app syncs with Inventory. The muffin item is visually flagged (Red text: "Sold Out"). The "Checkout" button is disabled until the conflict is resolved (item removed).

4.6 Payment and Voucher Application

Entry Point: Cart Footer.

1. **Voucher Selection:** User taps "Add Voucher." A modal lists available rewards (from Loyalty) and general coupons.
2. **Application:** User selects "\$5 Off."

- *Calculation:* The client sends a `dry_run` request to the Order API. The server responds with the new total. This prevents client-side math errors regarding tax/rounding.

3. **Payment Processing:**

- The app uses the Stripe SDK (or equivalent).
- *Optimistic Success:* Upon successful biometric auth (FaceID), the UI immediately shows the "Success" animation while the backend processes the webhook.

4.7 Order Tracking

Entry Point: Post-checkout or "Orders" tab.

1. **State Visualization:** A rigorous progress bar shows the order lifecycle:
 - Confirmed (Order received).
 - Preparing (Barista has tapped "Start" on KDS).
 - Ready (Order is on the counter).
2. **Live Updates:** The app uses a WebSocket connection or Silent Push Notifications to update this state in real-time.
3. **Pickup Action:** When state == READY, the screen displays a large "Pickup Number" (e.g., #92) for the user to show staff.

4.8 Store Discovery & Navigation

1. **List View:** Stores are sorted by a weighted algorithm: Status (Open > Closed) + Distance.
2. **Map View:** A map shows pins.
 - *Innovation:* The pin label isn't just the logo; it's the **Wait Time** (e.g., "5m", "15m"). This helps users distribute load away from busy stores.
3. **Navigation:** Tapping "Get Directions" opens the OS default (Apple Maps/Google Maps). We do not implement turn-by-turn navigation in-app.

4.9 Connect People (The "Social" Feature)

Entry Point: "Connect" Tab.

1. **Check-In:** User taps "I'm here."
 - *Privacy:* A modal asks, "Share with friends for 2 hours?"
2. **Status Broadcasting:** The user's avatar appears on their friends' "Connect" lists with the location "Downtown Cafe."
3. **Interaction:** A friend sees the status. They can tap "Gift a Coffee."
 - This opens a mini-menu. They pay. The user at the cafe receives a push: "Alex just bought you a coffee! Redeem it at the counter."

5. UI/UX Design System

The application adopts an "Editorial" design language. This aesthetic mimics high-end print

magazines or productivity tools like Notion: heavy reliance on typography, stark black/white contrast, and generous whitespace. It avoids the "cartoony" or "colorful blob" aesthetic common in food delivery apps.

5.1 Design Philosophy

- **Calmness:** The interface should not scream for attention. It is a tool.
- **Texture:** Use of "paper" whites (slightly warm) rather than clinical blue-whites.
- **Typography-First:** Hierarchy is established through font size and weight, not colored boxes or drop shadows.

5.2 Color System

The palette is derived from coffee tones, ensuring the food photography remains the hero.

Light Mode:

Token	Hex Value	Usage
Primary.Espresso	#362323	Main Headings, Primary Buttons, Active Tab Icons.
Secondary.Latte	#EEDAB5	Accent highlights, Selected states, Toggle switches.
Background.Paper	#FAFAFA	Main screen background. A warm, non-blinding white.
Surface.Card	#FFFFFF	List items, Modals, Input fields.
Text.Ink	#2D2A2A	Body text, subheadings.
Text.Muted	#6D6666	Captions, meta-data.
Semantic.Error	#855B5B	Error messages, Destructive actions.
Semantic.Success	#7D8D86	"Open" badges, Order confirmation.

Dark Mode Adaptations:

- Background.Paper shifts to #121212 (Not pure black).
- Surface.Card shifts to #1E1E1E.
- Primary.Espresso inverts to #FFFFFF for text legibility.
- Secondary.Latte desaturates to #C1A68D to reduce eye strain.

Contrast Compliance: All text pairings meet WCAG AA standards. For example, Text.Ink on Background.Paper has a ratio of >10:1.

5.3 Typography Scale

The system uses a classic editorial pairing: a characterful Serif for headings and a highly legible Sans-Serif for utility text.¹⁴

- **Headings:** *Playfair Display* (or similar high-contrast Serif).
- **Body/UI:** *Inter* (or system San Francisco/Roboto).

Token	Family	Size	Line Height	Weight	Application
Display.H1	Serif	32pt	40pt	Bold	Screen Titles (e.g., "Morning, Alex").
Display.H2	Serif	24pt	32pt	SemiBold	Section Headers (e.g., "Your Favorites").
UI.Title	Sans	18pt	24pt	Medium	Card Titles, Modal Headers.
UI.Body	Sans	16pt	24pt	Regular	Menu descriptions, settings text.
UI.Caption	Sans	14pt	20pt	Regular	Modifiers, secondary info.
UI.Micro	Sans	12pt	16pt	Medium	Tab labels, badges.
UI.Button	Sans	16pt	16pt	Bold	Call-to-Action buttons.

5.4 Spacing and Grid System

The layout enforces a strict **8-point grid** system.¹⁶ All margins and padding are multiples of 8.

- **Grid Base:** 8pt.
- **Margin (Screen Edge):** 16pt (Mobile standard) or 24pt (Editorial breathing room).
- **Gutter (Card Spacing):** 16pt.
- **Stack Spacing:**
 - Title to Subtitle: 8pt.
 - Subtitle to Content: 16pt.
 - Section to Section: 32pt or 48pt.

5.5 Component Library Specifications

Buttons:

- *Primary:* Primary.Espresso background, Surface.Card text. Height: 48px. Corner Radius: 8px (Soft square, avoiding the "pill" shape to look more serious/editorial).
- *Secondary:* Transparent background, 1px Primary.Espresso border.
- *Interaction:* On press, opacity reduces to 80%. No ripple effect (too "Material").

Cards (Menu Item):

- *Elevation:* 0. No drop shadows. Definition comes from a 1px border (#E0E0E0).
- *Corner Radius:* 12px.
- *Layout:* Image (80x80px) on the left (or right), Text vertically centered.

Tab Bar:

- Background: Translucent/Blur material (UIBlurEffect on iOS).
- Icons: Lucide style, 24x24px.
- Active State: Icon fills with Primary.Espresso. No text labels if possible, or very subtle labels.

Icon System:

- **Source:** Lucide Icons (Clean, consistent stroke width of 1.5px).
- **Usage:** Icons are functional (Cart, Profile, Back, Close). They are *not* decorative. We do not use icons to represent "Coffee" or "Croissant" in the menu; we use text or photography.
- **Fallback:** If a specific icon is missing, default to a generic shape (e.g., a circle) or text-only.

5.6 Accessibility

- **Dynamic Type:** All text elements must use scalable units (sp/pt) and support OS-level font scaling up to 200%. Layouts must use Auto Layout to expand vertically as text grows.
- **Touch Targets:** All interactive elements (Buttons, Icons) must have a minimum hit-test area of 44x44 points, even if the visual icon is smaller.

6. Detailed Feature Specification: The Cart Screen

The Cart Screen is the transactional funnel's critical point. It requires precise state management and error handling.

6.1 UI Layout

The screen is presented as a "Sheet" or a dedicated view.

- **Header:** "Your Order" (H2 Serif).
- **Scroll View:**
 - **Grouped Items:** Items are grouped by Store (though effectively the cart only supports one store at a time).
 - **Cart Item Row:**
 - Left: Quantity Stepper. A distinct component [- 2 +].
 - Center: Item Name (Bold) + Modifier String (Caption, Grey).
 - Right: Price (e.g., "\$12.50").
- **Footer (Sticky):**
 - **Upsell:** A single line text "Forgot a cookie? +\$2.00" (AI driven).
 - **Voucher Row:** "Promo Code or Reward >" (Tapping opens selection).
 - **Summary:** Subtotal, Tax, Delivery Fee, **Total**.
 - **Checkout Button:** Full width, Primary.Espresso. Label: "Checkout • \$15.50".

6.2 Interaction Model & Logic

- **Editing Modifiers:** Tapping the text of an item does *not* just highlight it. It opens the **Item Detail Modal** pre-populated with the current choices. The user can change "Oat Milk" to "Whole Milk" and tap "Update Order."
 - *System Logic:* This replaces the CartItem in the array and recalculates the price.
- **Quantity Change:**
 - Incrementing (+) triggers an **Optimistic UI Update**. The number changes instantly. The API call happens in the background.
 - *Failure:* If the API returns "Insufficient Stock," the number reverts, and a Toast error appears: "Sorry, only 2 left."
- **Removal:** Reducing quantity to 0 transforms the row into a "Delete" confirmation state or simply removes it (with an "Undo" snackbar).

6.3 Edge Cases and Error Handling

- **Price Drift:** If the price of an item changes between the time it was added and the time the cart is viewed (e.g., Happy Hour ends), the Cart API response flags the item.
 - *UI:* A banner appears at the top: "Note: Some prices have changed." The old price is struck through, new price shown.
 - **Stock Depletion:** If an item goes OOS while in the cart, the item row turns semi-transparent. The "Checkout" button is disabled. An error message appears inline: "Currently Unavailable." The user must remove it to proceed.
-

7. Feature Specification: Store & Map Experience

7.1 Store List Sorting Logic

The "Find a Cafe" list is not a static database dump. It utilizes a weighted sorting algorithm:

1. **Filter:** Active == True.
2. **Primary Sort: Functional Availability.**
 - Bucket A: Open Now AND Within Delivery Radius (if mode is delivery) OR Within 5km (if mode is pickup).
 - Bucket B: Open Now but distant.
 - Bucket C: Closed.
3. **Secondary Sort:** Linear Distance (Haversine formula) from User Location.

7.2 Store Detail Screen

- **Header:** Hero image of the specific location interior.
- **Info Block:** Address, Status Badge ("Open until 9pm"), Distance ("0.5mi away").
- **Features:** A row of icons/badges: "WiFi", "Outdoor Seating", "Drive-thru."
- **Action:** "Order Here" (Primary Button).

7.3 Directions & Navigation

The app creates a seamless handoff to native maps.

- **Interaction:** Tapping the Map Pin or the Address on the Detail screen.
 - **Logic:** The app constructs a universal link (e.g., <http://maps.apple.com/?daddr=lat,long>) which opens Apple Maps on iOS and Google Maps on Android.
 - **Why:** Building turn-by-turn navigation inside a cafe app is feature bloat and a maintenance liability.
-

8. Feature Specification: AI-Assisted Personalization

This feature addresses the "Paradox of Choice" and friction. It is a **Context-Aware Recommender System** designed to shortcut the ordering process.¹⁷

8.1 The "Quick Order" Modal

Trigger: This logic runs immediately upon App Launch (Cold Start).

The Algorithm (Simplified Rule-Based):

The system evaluates a "Context Score" for the user's history.

- *Input:* Current Time (e.g., 8:00 AM), Current Location, Day of Week.
- *Historical Pattern:* Look for orders placed at this Time +/- 1 hour and Location +/- 1km.
- *Threshold:* If >3 matching orders in the last 30 days are found for the *same items*, confidence is High.

The Experience:

- If Confidence > 80%: A modal overlays the screen.
 - **Headline:** "The usual, Alex?"
 - **Content:** "Large Oat Latte • Pickup at Downtown."
 - **Action:** "Order Now (\$6.50)" -> Tapping this creates the Cart and deep-links directly to the **Checkout Screen**, bypassing the menu entirely.
- *Cold Start / Low Confidence:* If the user is new or history is erratic, the modal is suppressed. The Home screen instead displays "Trending at" section based on aggregate store sales data.

8.2 Privacy and Explanation

- **Transparency:** The modal includes a micro-copy line: "Suggested based on your weekday morning history."
 - **Opt-Out:** The User Settings includes a toggle: "Enable Smart Suggestions."
 - **Failure Handling:** If the user dismisses the modal (X) three times consecutively, the system enters a "Cool-down" mode and stops showing it for 7 days to avoid annoyance.
-

9. Feature Specification: Role-Based Access Control

(RBAC)

The application code is shared, but the interface adapts based on the authenticated user's Role Claim in their JWT.¹⁹

9.1 The Customer Role (Default)

- **Scope:** Can READ Public Menus, WRITE Own Orders, READ Own Profile.
- **UI:** Standard consumer interface defined in this spec.

9.2 The Staff Role (Barista)

- **Authentication:** Staff login via a specific "Employee Portal" button using corporate credentials.
- **UI Transformation:** The "Order" tab is replaced by a **KDS (Kitchen Display System)** view.
 - *View:* A Kanban board of "Incoming," "Preparing," "Ready."
 - *Capabilities:* Can change Order State. Can toggle Item Availability (e.g., mark "Croissant" as OOS). This updates the global inventory instantly.
 - *Restrictions:* Cannot edit Menu Prices. Cannot issue Vouchers.

9.3 The Admin Role (Manager)

- **Scope:** Full access.
- **UI Transformation:** Adds a "Dashboard" tab.
 - *View:* Real-time sales graph, User analytics.
 - *Capabilities:* Can issue refund authorizations. Can edit Store Hours.

10. Non-Goals, Risks, and Trade-offs

10.1 Non-Goals

- **Table Reservations:** We explicitly exclude table booking logic. This adds massive complexity (time slot management, table inventory) and is ill-suited for a high-turnover fast-casual cafe model.
- **In-App Chat:** We will not build a chat system between users or staff. Communication is handled via automated status updates and transactional SMS.
- **Driver App:** We assume delivery fulfillment is handled either via a 3rd-party API integration (e.g., Uber Direct, DoorDash Drive) or that drivers use a separate, dedicated operational app. This spec covers the *Consumer* app only.

10.2 Technical Trade-offs

- **Server-Side Cart vs. Client-Side:** We choose **Server-Side**.
 - *Trade-off:* Requires a network call for every quantity change (latency).

- *Benefit*: Persistence (cart saves across devices) and Inventory Truth (we validate stock immediately).
 - *Mitigation*: Use "Optimistic UI" patterns to mask the network latency.
 - **Approximate vs. Exact Location Sharing**: For the social feature, we choose **Approximate**.
 - *Trade-off*: You can't see exactly which table a friend is at.
 - *Benefit*: Massive gain in user trust and privacy safety.
-

11. Conclusion

This specification defines a mobile application that is robust, scalable, and focused on high-value utility. By strictly bounding the feature set to Ordering, Loyalty, and Contextual Intelligence, CafeOS avoids the bloat that plagues modern "Super Apps." The architectural decisions—specifically the use of Server-Side state management for financial integrity combined with Optimistic UI for perceived speed—ensure a product that is both reliable for the business and delightful for the user. The detailed data models and design tokens provided herein serve as the immediate blueprint for engineering and design teams to commence the build phase.

Nguồn trích dẫn

1. UI/UX Design Trends in Mobile Apps for 2025 | Chop Dawg, truy cập vào tháng 1 16, 2026, <https://www.chopdawg.com/ui-ux-design-trends-in-mobile-apps-for-2025/>
2. Mobile UX design examples from apps that convert (2025) - Eleken, truy cập vào tháng 1 16, 2026, <https://www.eleken.co/blog-posts/mobile-ux-design-examples>
3. Building Lightning-Fast UIs: Implementing Optimistic Updates with React Query and Zustand, truy cập vào tháng 1 16, 2026, <https://medium.com/@anshulkahar2211/building-lightning-fast-uis-implementing-optimistic-updates-with-react-query-and-zustand-cfb7f9e7cd82>
4. How to Use the Optimistic UI Pattern with the useOptimistic() Hook in React - freeCodeCamp, truy cập vào tháng 1 16, 2026, <https://www.freecodecamp.org/news/how-to-use-the-optimistic-ui-pattern-with-the-useoptimistic-hook-in-react/>
5. (PDF) The Impact of Mobile Order Ahead Apps on QSR Queuing - ResearchGate, truy cập vào tháng 1 16, 2026, https://www.researchgate.net/publication/367117506_The_Impact_of_Mobile_Order_Ahead_Apps_on_QSR_Queueing
6. How Apps Know Your Food Is 32 Minutes Away: Demystifying ETA Prediction with Machine Learning | by Ranjansiya | Nov, 2025 | Medium, truy cập vào tháng 1 16, 2026, <https://medium.com/@ranjansiya07/how-apps-know-your-food-is-32-minutes-away-demystifying-eta-prediction-with-machine-learning-a57e02ddbd8c>

7. Calculating order wait time - - Developer guide - Toast platform docs -, truy cập vào tháng 1 16, 2026,
<https://doc.toasttab.com/doc/devguide/calculatingOrderWaitTime.html>
8. The interplay between social media use and problematic internet usage: Four behavioral patterns - PMC - NIH, truy cập vào tháng 1 16, 2026,
<https://pmc.ncbi.nlm.nih.gov/articles/PMC10163648/>
9. How Do I Build Real-Time Location Sharing Features? - Mobile app developers, truy cập vào tháng 1 16, 2026,
<https://thisisglance.com/learning-centre/how-do-i-build-real-time-location-sharing-features>
10. Privacy-First Design | AI Design Patterns - aiux, truy cập vào tháng 1 16, 2026,
<https://www.aiuxdesign.guide/patterns/privacy-first-design>
11. Create and Manage Menus, Menu Groups, and Subgroups - Toast Central, truy cập vào tháng 1 16, 2026,
<https://central.toasttab.com/s/article/Menus-Menu-Groups-and-Subgroups>
12. Designing a flexible and comprehensive restaurant menu database schema?, truy cập vào tháng 1 16, 2026,
<https://softwareengineering.stackexchange.com/questions/453121/designing-a-flexible-and-comprehensive-restaurant-menu-database-schema>
13. 15 Ecommerce Checkout & Cart UX Best Practices for 2026, truy cập vào tháng 1 16, 2026,
<https://www.designstudiouiux.com/blog/ecommerce-checkout-ux-best-practices/>
14. Recommendations to create the typography system for your design system, truy cập vào tháng 1 16, 2026,
<https://www.designsystemscollective.com/recommendations-to-create-the-typography-system-for-your-design-system-cb2aa25978ca>
15. Best google font pairings for UI design in 2025 | by Matt Medley | Bootcamp - Medium, truy cập vào tháng 1 16, 2026,
<https://medium.com/design-bootcamp/best-google-font-pairings-for-ui-design-in-2025-ba8d006aa03d>
16. The 8pt Grid System: A Simple Guide to Consistent UI Spacing - Rejuvenate Digital, truy cập vào tháng 1 16, 2026,
<https://www.rejuvenate.digital/news/designing-rhythm-power-8pt-grid-ui-design>
17. SMART Restaurant ReCommender: A Context-Aware Restaurant Recommendation Engine - MDPI, truy cập vào tháng 1 16, 2026,
<https://www.mdpi.com/2673-2688/6/4/64>
18. Building a Context-Aware Street Food Recommender System Using Kiro, truy cập vào tháng 1 16, 2026,
<https://builder.aws.com/content/37PtoEoTegcKiD6Ogv0A37w1YIM/building-a-context-aware-street-food-recommender-system-using-kiro>
19. Role-Based Access Control (RBAC) | Couchbase Docs, truy cập vào tháng 1 16, 2026,
<https://docs.couchbase.com/server/current/rest-api/rbac.html>
20. Reactive REST API response filtering based on user roles - Stack Overflow, truy cập vào tháng 1 16, 2026,

<https://stackoverflow.com/questions/51763662/reactive-rest-api-response-filtering-based-on-user-roles>